GEK-83814

DRIVE SYSTEM INSTRUCTIONS

LADDER DIAGRAM SUPPORT SYSTEM SOFTWARE Version I

DRIVE SYSTEMS DEPARTMENT
SALEM, VA. 24153

GENERAL@ ELECTRIC

LADDER DIAGRAM SUPPORT SYSTEM SOFTWARE

GER-83814

ISSUED: July, 1983

The information herein does not purport to cover all details or variations in equipment nor to provide for every possible contingency to be met in connection with installation, operation, and maintenance. Should further information be desired or should particular problems arise which are not covered sufficiently for the purchaser's purposes, the matter should be referred to General Electric Company, Drive Systems Department, USA.

NOTICE

A. This Manual and the Software discussed herein incorporate confidential proprietary information and/or copyrighted material of General Electric Company and except as authorized in writing by General Electric shall not.be disclosed to any third party.

B. No title to or ownership of the Software has been transferred and the Software remains the property of General Electric Company.

C. Except as provided below, the Software, including any subsequent updates and any part thereof may be used on only the single computer CPU on which the Software is first installed.

1. In the event that an equipment malfunction occurs in the above single computer CPU causing the Software to become inoperable on such single computer CPU, the Software (or copies therefor) may be used on another single computer CPU on a temporary basis during such malfunction.

D. The Software may be copied, in whole or part (with the proper inclusion of any General Electric copyright or proprietary notice) only as necessary for its efficient use on such single computer CPU.

E. This manual may not be copied in whole or in part. If additional copies are required, they may be obtained from General Electric Company.

Copyright © 1983 by General Electric Company.
Published only in a limited, copyright sense and all rights, including trade secret rights, are reserved.

TABLE OF CONTENTS

CHAPTER 1 - INTRODUCTION

Hardware Requirements 2
System Overview 2
-Programs. 2
Editor 2
Compiler 2
Loader 2
Documentatin Generator 2
Files 3
Source File (.SRC) 3
Listing File (.LIS). 3
Names File (.NAM). 3
I/O Definition File (.IOD) 3
Object File (.OBJ) 3
Register File (.REG) 3
Documentation File (.DOC). 3
Error Files (.ERC, .ERL, .ERD) 3
Summary of Operating Procedures 5
Program Development. 5
Debug Testing. 5
Documentation Generation 5
Field Release. 5
Field Updating 5
Archiving. 6

CHAPTER 2 - HOW TO USE THE LDSSS EDITOR

Terminal Display and Keyboard Functions 7
Operating Instructions. 13
Control Functions 13
Keypad Control Functions 13
VT100 control Key Functions. 15
Edit Function Control Keys 15
Special Functions in LDSSS Editor. 16
Relay Editing 18
Diagram Editing. 18
Signal Name Editing. 22
Comment Editing 23
Comment Edit Function Keys. 24
Special Comments 25
Inputs 27
Old Source file. 27
Comment Reintegration File 27

CHAPTER 3 - SERIES SIX COMPILER

Inputs 28
Outputs 28
Operating Instructions. 29

CHAPTER 4 - DOCUMENTOR

Operation 30
Input Used 30
outputs 31

CHAPTER 5 - CCMA1 LOADER

Operation 32

CHAPTER 6 - INSTALLATION PROCEDURE. 34

Appendix - Examples of all files used or generated by the LDSSS software. 35

SERIES SIX FIELD SUPPORT SOFTWARE

GEK-83814

ISSUED: July, 1983

The information herein does not purport to cover all details or variations in equipment nor to provide for every possible contingency to be met in connection with installation, operation, and maintenance. Should further information be desired or should particular problems arise which are not covered sufficiently for the purchaser's purposes, the matter should be referred to General Electric Company, Drive Systems Department, USA.

NOTICE

A. This Manual and the Software discussed herein incorporate confidential proprietary information and/or copyrighted material of General Electric Company and except as authorized in writing by General Electric shall not be disclosed to any third party.

B. No title to or ownership of the Software has been transferred and the Software remains the property of General Electric Company.

C. Except as provided below, the Software, including any subsequent updates and any part thereof may be used on only the single computer CPU on which the Software is first installed.

1. In the event that an equipment malfunction occurs in the above single computer CPU causing the Software to become inoperable on such single computer CPU, the Software (or copies therefor) may be used on another single computer CPU on a temporary basis during such malfunction.

D. The Software may be copied, in whole or part (with the proper inclusion of any General Electric copyright or proprietary notice) only as necessary for its efficient use on such single computer CPU.

E. This manual may not be copied in whole or in part. If additional copies are required, they may be obtained from General Electric Company.

Copyright © 1983 by General Electric Company.
Published only in a limited, copyright sense and all rights, including trade secret rights, are reserved.

CHAPTER 1
INTRODUCTION

The Ladder Diagram Support System Software (LDSSS) provides the tools, procedures, and security to efficiently create, document, test, distribute, and archive Series Six progams. The Series Six Support System features:

1. The ability to efficiently create and edit Series Six programs.

2. A file system that allows retrieval of standard patterns and parts of existing programs to use as the basis for new programs. This eliminates the need to re-enter data and provides the capability to resolve conflicts in I/O signal and register usage between patterns.

3. Translation to target-machine-loadable media.

4. The ability to generate documentation, including:

   a. standard Series Six diagrams with the addition of signal names, comments, page headings, and I/O information,

   b. cross reference listings and where-used data on signal names, signal addresses, and registers,

   c. both low cost (i.e., line-printer) and higher cost, higher quality documentation.

5. Program testing which is independent of the target machine hardware, thus minimizing the need for factory staging due to software testing.

6. A mechanism to update application documentation and files to "as installed" form. Updates could be necessary due to program changes defined in load media form or changes in the hardware I/O configuration.

HARDWARE REQUIREMENTS

The LDSSS runs on a Digital Equipment Corporation VAX 11/780 using the following resources (which can be timeshared with other applications):

* VT100 terminal for program editing.

* Disk space for on-line program storage.

* Magnetic tape for archival storage.

* Printer for hardcopy documentation.

The system also requires a dedicated Series Six programmable controller system with a CCM version 1 communication card and a Series Six Programming Data Terminal (PDT). The Series Six system is used for program debugging and distribution. An RS232 switch box is also needed to switch the VAX line between the VT100 and the Series Six.

SYSTEM OVERVIEW

The LDSSS consists of four programs: the Editor, the Compiler, the Loader, and the Documentation Generator. These programs use and/or generate seven different types of files: source files, object files, I/O Definition files, listing file, documentation files, names files, and error files. Figure 1 shows the relationships between the LDSSS programs and files, each of which are summarized below.

Programs

Each LDSSS program is described in greater detail elsewhere in this manual. The editor is described in Chapter 2, the compiler is described in Chapter 3, the loader is described in Chapter 4, and the documentation generator is described in Chapter 5.

Editor The Editor allows you to edit Series Six programs in relay' diagram form using a VAX VT100 terminal. Programs can include comment lines and mnemonic names for I/O signals and registers. An important feature of the Editor is the ability to copy parts of other Series Six programs to eliminate the need to re-enter this data.

Compiler The Compiler converts source code created by the Editor into object code that can be loaded into and executed by a Series Six programmable controller. The Compiler can also reverse translate object code back into source code.

Loader The Loader provides a means to transfer object programs to and from the Series Six CPU. Programs are loaded and retrieved via an asynchronous communication link. The link features check sums and automatic retransmission to ensure reliable communications.

Documentation Generator The Documentation Generator provides quality documentation for Series Six programs. Documentation includes page headers and footers, signal names and addresses, text comments, forward and backward cross references, signal descriptions, and a comprehensive cross reference table.

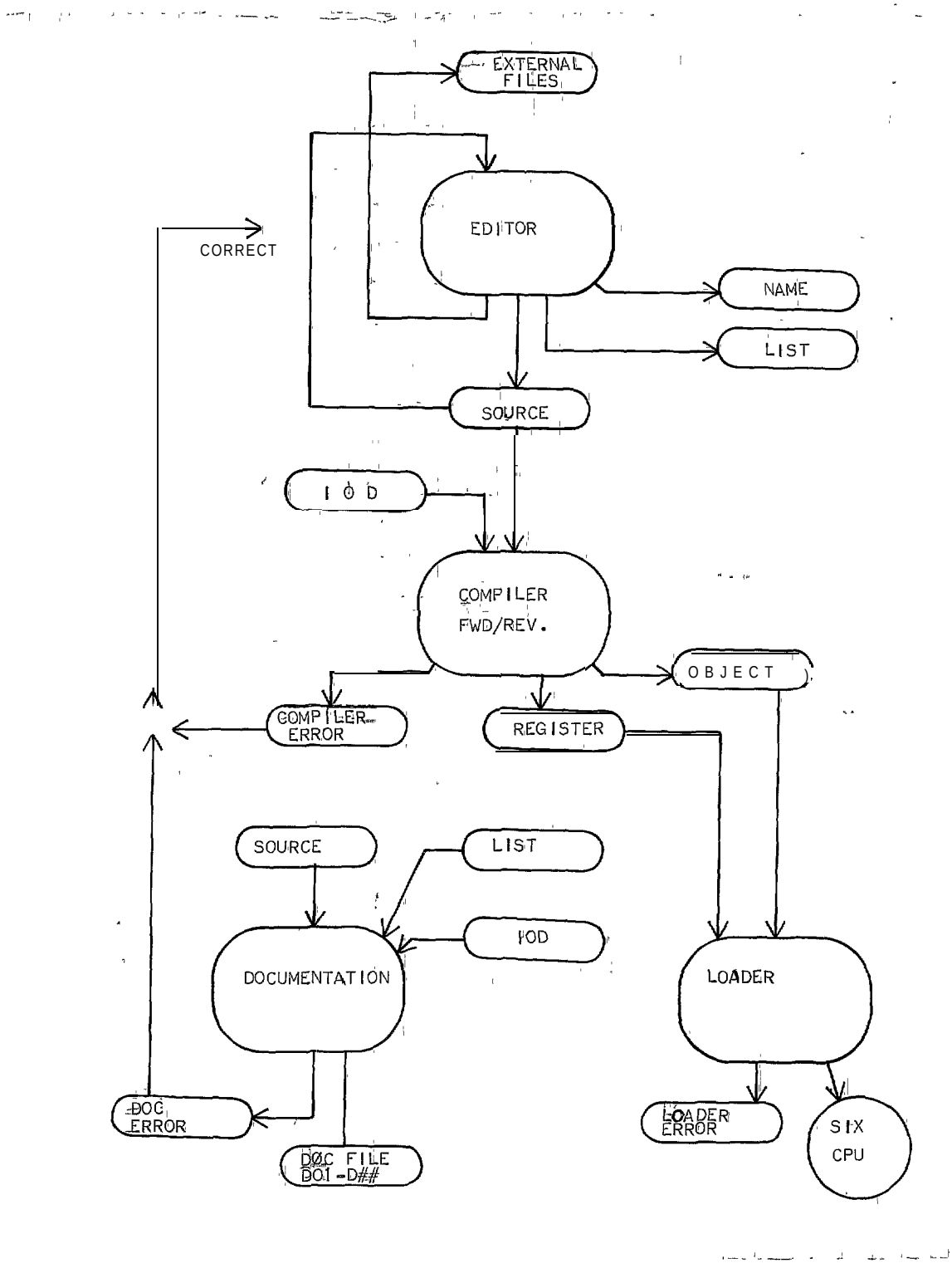


Figure 1
File Flow Chart

Files

The LDSSS generates and/or uses the seven types of files described below. The letters in parentheses indicate how the file type is identified on the VAX system. For instance, FILENAME.SRC is a source file, while FILENAME.OBJ is an object file.

Source File (.SRC). A source file contains a Series Six program in a compressed form for archival storage. The program can include comment lines and mnemonic names for signals and registers. A source file is generated by the Editor and used by the Compiler to produce an object file. A source file can also be generated by the compiler through reverse translation of an object file.

Listing File (.LIS). A listing file contains a Series Six program in relay diagram form, which can include comments and mnemonic signal names. The file is used by the documentation generator and can also be printed out to provide readable hard copy during program development.

Names File (.NAM). The names file is an optional output of the LDSSS Editor. It contains a list of all signal names used within a program. The file is useful for creating an I/O Definition file using the VAX EDT editor.

I/O Definition File (.IOD). An I/O Definition file contains a list of mnemonic names and the associated I/O signal or register hardware address. The file may also contain description data. The file is generated using the VAX EDT editor.

Object File (.OBJ). An object file contains a Series Six program in a form that can be loaded and executed by a Series Six CPU. Object files are generated by the Compiler and can be loaded to or retrieved from a Series Six CPU by the Loader.

Register File (.REG). An object file containing the register table. It will be down loaded to Series Six CPU by the loader program.

Documentation File (.D**). Documentation file(s) contain quality documentation including page headers and footers, signal names and addresses, text comments, forward and backward cross references, signal descriptions, and a comprehensive cross reference table. The document file is divided into many small files. The file extensions will be .D01, .D02,•.• Dnn. The last file (.Dnn) will contain table of contents and cross reference data.

Error Files (.ERC, .ERL, .ERD). Error files are generated by the Compiler (.ERC), the Loader (.ERL), and the Documentation Generator (.ERD). If no errors are found, this file will be empty. If errors are found, this file will contain error messages.

Examples of all files are shown at the end of this manual.

SUMMARY OF OPERATING PROCEDURES

This section contains a summary of how the LDSSS is used to develop, debug, document, release, revise, and archive Series Six software. Detailed instructions for operation of each program will be found later in this manual.

Program Development

Program development is done using the LDSSS Editor, the VAX EDT Editor, and the LDSSS Compiler. First, the LDSSS Editor is used to create and edit the source file. If desired, a listing file can be generated and printed for a quick hard copy listing of the program. The name and address file will normally begin with an I/O data file from the General Electric Proposition/Requisition Elementary System. Additional names and addresses for registers and internal signals are added using the VAX EDT editor. The Compiler is used to translate the source file and name & address file into an object file. This process of editing and compiling is repeated until the program compiles error free.

Debug Testing

Debug testing of the software is performed on a Series Six system. The LDSSS Loader is used to load a program object file from the VAX into the Series Six CPU. The Series Six Program Development Terminal (PDT) is then used to monitor the execution of the program in the Series Six CPU.

Documentation Generation

At the end of debug testing, the Documentation Generator is used to generate quality documentation.

Field Release

After debug testing, the program object file in the Series Six CPU can be written to cassette tape using the Series Six PDT. The tape and documentation can then be released to the field or used on another Series Six system.

Field Updating

Program changes in the field are made using a Series Six PDT. After the program has been revised, the documentation is updated as follows:

1. The revised program is received from the field on cassette tape.
2. The cassette tape is loaded into the Series Six CPU using the PDT.
3. The LDSSS Loader is used to load the object file into the VAX.
4. The LDSSS Compiler is used to reverse translate the object file back into source.
5. The VAX EDT editor is used to update the I/O Definition file as necessary.
6. The LDSSS Documentation Generator is rerun to generate updated documentation.

A possible alternative to steps 1 and 2 is to use a dial-up communications line between the Series Six CPU and the VAX in step 3.

Archiving

A Series Six program is archived by writing the source file and I/O Definition file to magnetic tape. The other files can be regenerated from these two files as necessary.

CHAPTER 2
HOW TO USE THE LDSSS EDITOR

The LDSSS Editor allows you to edit Series Six source programs on the VAX. The LDSSS Editor is similar to the editor provided on the Series Six PDT with the following exceptions:

1. The LDSSS Editor is designed for high speed input. It supports a left to right input sequence and minimizes operator keystrokes.

2. I/O signals and registers may be identified by mnemonics as well as by hardware addresses. This makes programs easier to read and allows programming to proceed prior to finalizing hardware address allocations.

3. Text comments can be included in the relay diagram, making programs easier to read.

4. Programs are stored on the VAX disk, eliminating the need to load programs from data cartridge.

5. Existing programs are easy to copy and modify, which in many cases eliminates the need to re-enter data. In particular, signal names can be modified without re-entering the relay diagram.

6. The VAX system provides a means to permanently archive Series Six programs.

TERMINAL DISPLAY AND KEYBOARD FUNCTIONS

The LDSSS Editor uses a DEC VT100 terminal to display and modify Series Six programs in relay diagram form. Relay diagrams are displayed on an 8 x 10 grid as shown in Figure 2. The terminal can also be used to display comments of up to 24 lines of 80 characters. The bottom line of the display is used for operator dialog and error messages.

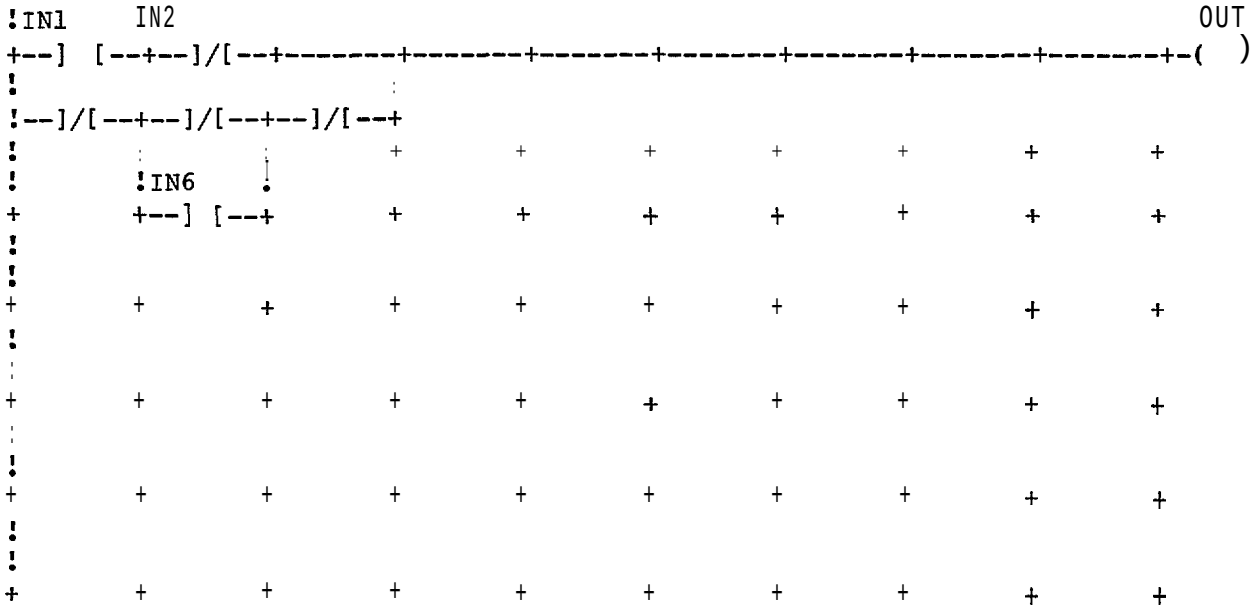
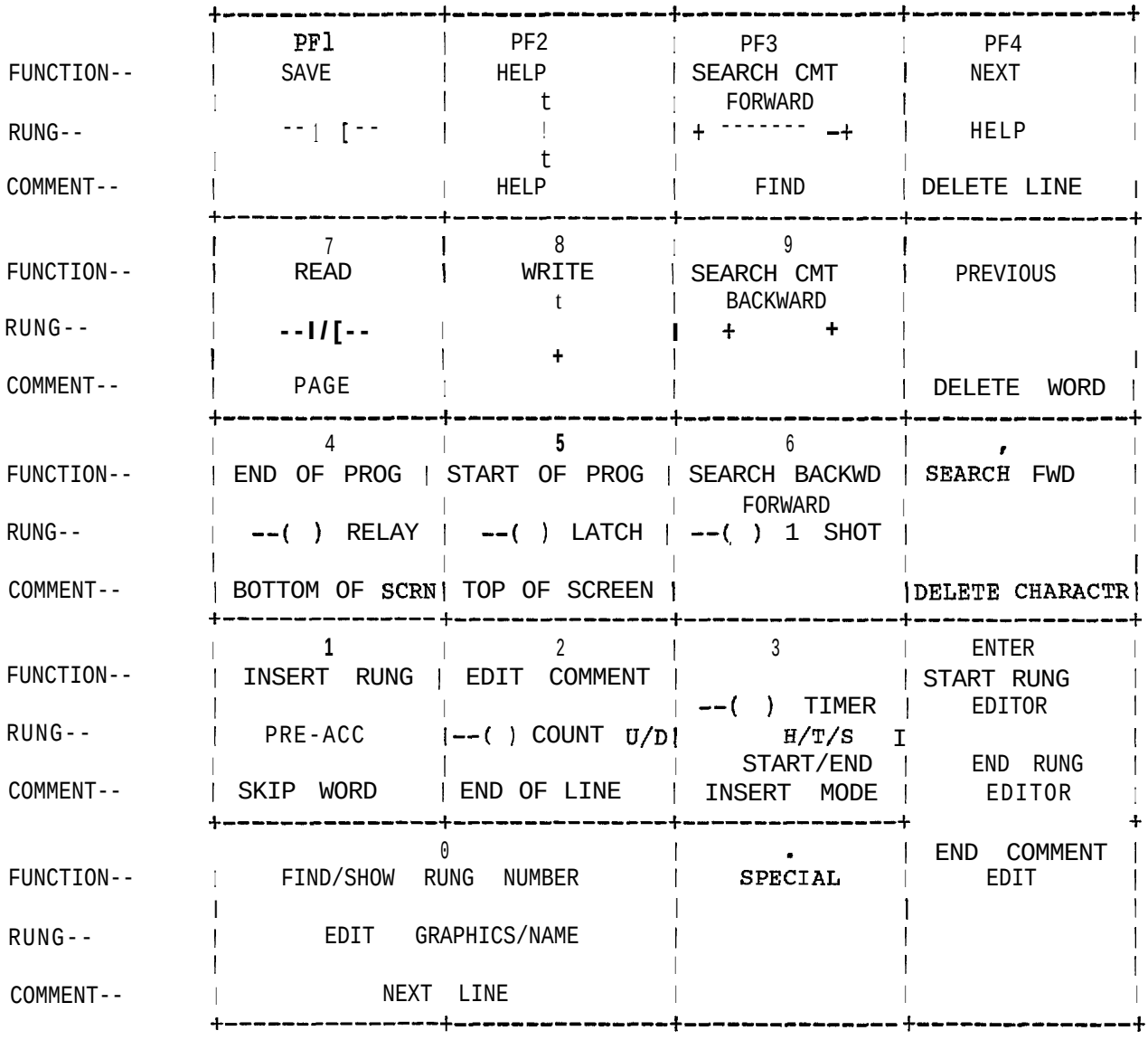


Figure 2.
VT100 Display of LDSSS Editor Relay Diagram

The LDSSS Editor uses the standard VT100 keyboartd with the numeric keypad functions redefined as shown in Figure 3. The more common edit functions are available through this redefined keypad with a single keystroke. The less frequently used functions are input by depressing the SPECIAL (decimal point) key on the numeric keypad followed by the sequences shown in Figure 3.

The other functions are available by using the VT100 CONTROL key: (1) Pressing the CONTROL key and then R refreshes the screen; (2) Pressing the CONTROL key and then Y causes an exit from the editor without saving the updated file.



FUNCTION	SPECIAL +	FUNCTION	SPECIAL +
Add Comments	AC	Rename Signal	RN
Delete Current Rung	D	Search Comment String Backward	CB
Delete Multiple Rungs	DM	Search Comment String Forward	CF
Create Ladders (List)	P	Search Significant Name Backward	SB
Create Signal Names	PN	Search Significant Name Forward	SF

Figure 3
VT100 Numeric Keypad Functions for LDSSS Editor

KEY PAD FOR EDIT FUNCTION MODE

SAVE	'HELP	SEARCH COMMENT FORWARD	NEXT RUNG
PF1	PF2	PF3	PF4
READ FILE	WRITE FILE	SEARCH NAME BACKWARD	PREVIOUS RUNG
7	8	9	--
DISPLAY FIRST RUNG	DISPLAY LAST RUNG	SEARCH NAME BACKWARD	SEARCH NAME FORWARD
4	5	6	,
INSERT RUNG	EDIT COMMENT		START RUNG EDIT
1	2	3	
FIND/SHOW RUNG NUMBER		SPECIAL	
0		.	ENTER

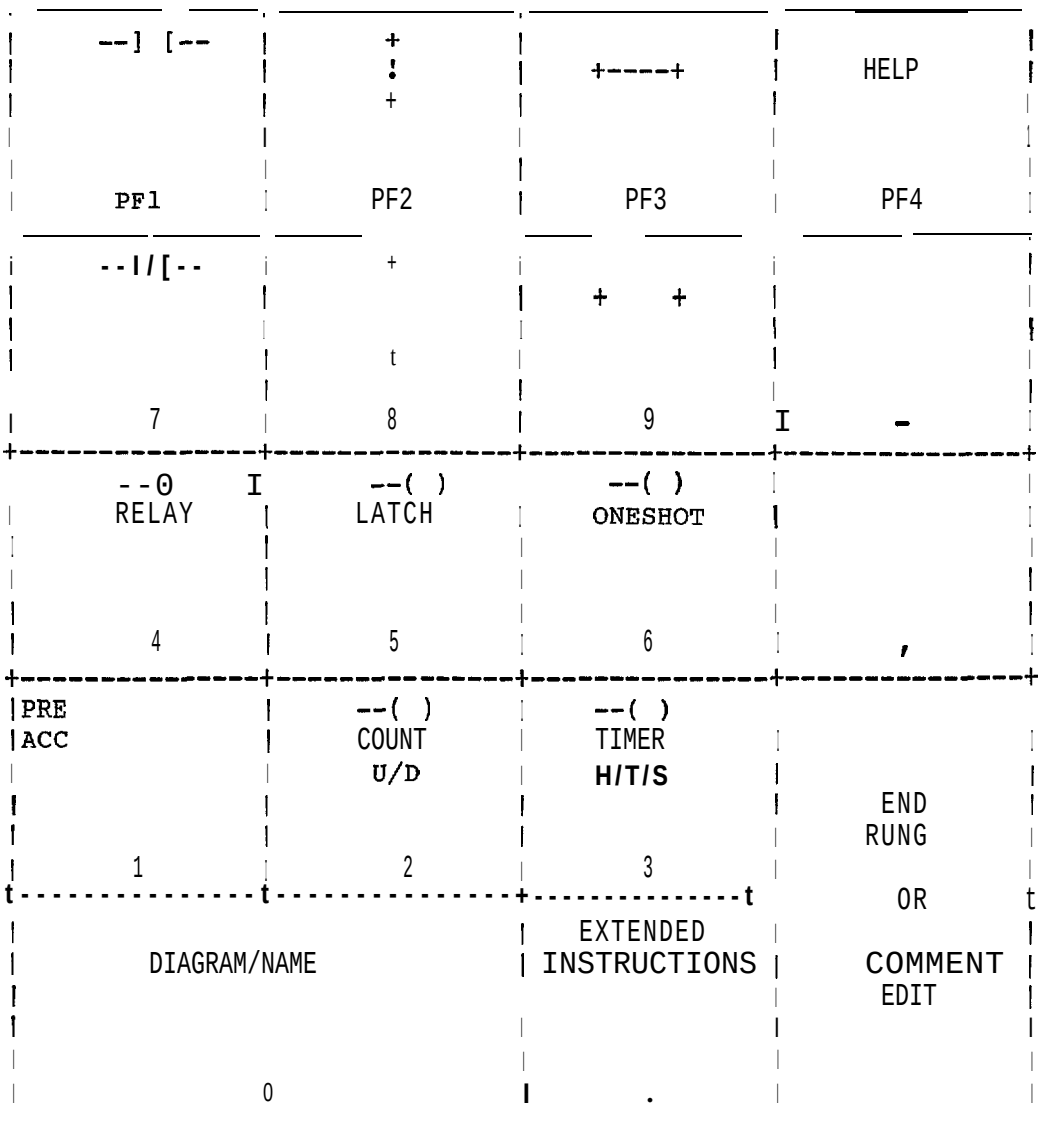
SPECIAL FUNCTIONS

AC - Add Comment
CB - Search comment string backward
CF - Search comment string forward
D - Delete current rung
DM - Delete multiple rungs
P - Pring ladder diagram
PN - Print names
RN - Rename signal

Other Keys

Control - R Repaint screen
Control - Y Abort LDSSS Editor

KEY PAD FOR EDIT RUNG MODE



Other Keys

Control - R	Repaint the screen
Control - Y	Abort rung edit and return to function mode
Delete	Rubout character
Backspace	Move cursor to column 1 for current line
Return	Move Cursor to column 1 of next line
Space Bar	Move cursor to next position
Arrow Keys	Move cursor in the screen

KEYPAD FOR THE COMMENT EDIT MODE

	HELP	FIND	DELETE LINE
PF1	PF2	PF3	PF4
PAGE		FIND NEXT	DELETE WORD
7	8	9	I -
BOTTOM OF SCREEN	TOP OF SCREEN		DELETE CHARACTER
4	5	6	,
NEXT WORD	END OF LINE	START/END INSERT MODE	END COMMENT EDITOR
1	2	3	
NEXT LINE			
0		.	ENTER

Other Keys

Control - R	Repaint the screens
Control - Y	Abort Comment edit and return to function mode
Delete -	Rubout character
Backspace -	Move curser to column 1 of current line
Return -	Add new line
Arror Keys -	Move cursor in the screen

OPERATING INSTRUCTIONS

The LDSSS Editor is activated by typing the following on the VT100:

RSE (Note: All inputs should be followed by the RETURN key.)

The editor will then prompt for the name of the program file to be edited:

FILE NAME?

Enter the filename with no extension. The LDSSS editor will automatically add the extension .SRC. If a file already exists with the name specified, that file will be opened for editing. A normal exit at the end of the edit session will generate a new version of the file. If no file exists with the name specified, the following message will be displayed:

FILE DOES NOT EXIST

Editing will begin with an empty program. A normal exit at the end of the edit session will begin at START OF DATA with a request for a control function to be performed. Control functions are covered in the next section of this manual.

IMPORTANT: To edit .SRC file, use only RSE, DO NOT USE EDT:

Control Functions

Control functions are used to display different parts of the relay diagram, to search for signal names, to initiate editing of relays and comments, to read and write files, and to terminate the edit session. The most common control functions are accessed by use of the VT100 numeric keypad shown in Figure 3.

An additional set of less frequently used control functions are accessed by use of the SPECIAL key on the numeric keypad in conjunction with the alphanumeric keyboard. These special functions are listed on the previous 4 pages.

Keypad control Functions The following functions are available by pressing the indicated key on the VT100 numeric keypad.

INSERT RUNG (Key 1)
Allows the user to edit a new relay to be inserted immediately following the current relay or comment.

EDIT COMMENT (Key 2)
Allows the user to edit a new comment to be inserted immediately above the current relay or comment.

READ FILE (Key 7)

Allows insertion of relays and associated comments from another program source file into the file being edited. The user is prompted for the name of the program source file from which the data is to be taken. The relays and comments are inserted immediately following the current relay. The user will be prompted as shown below:

```
OLD FILE NAME?
REPLACE "/" IN SIGNAL NAME WITH:
REPLACE" { } " IN COMMENT WITH:
```

If the user enters non blank characters in response to replace "/" prompt, then the "/" will be replaced by the characters specified by the user. The replacement character string can be one to seven characters long.

Example:

Replace "/" with "1 2 3"

Old Signal Name	New Signal Name
A/	A123
/A	123A
A/B	A123B
AB/CD	AB123CD
ABC/DEF	*

If the final name becomes more than 7 characters, it will be replaced by "*".

If the user enters non blank characters in response to replace "{ }" prompt, the "{ }" will be replaced by the character string specified by the user. The replacement character string can be 1 to 80 characters long with imbedded blanks.

Example:

Replace "{ }" with "CHARACTER REPLACEMENT EXAMPLE".

Old String	New String
TEST1 { }	TEST1 CHARACTER REPLACEMENT EXAMPLE
{ } TEST2	CHARACTER REPLACEMENT EXAMPLE TEST2
INSERT { } HERE	INSERT CHARACTER REPLACEMENT EXAMPLE HERE

WRITE FILE (Key 8)

Allows the user to create a new program source file and copy a specified number of relays and comments to that file. The user is prompted for the name of the new file and the number of relays and comments to copy. Copying begins with the current relay or comment.

SAVE (Key PF1)

Terminates the edit session after updating the program file.

HELP (Key PF2)

Displays the numeric keypad diagram shown in Figure 3.

VT100 Control Key functions The VT100 CTRL (Control) key can be used in conjunction with two alphabetic keys to perform the following function:

CONTROL Y

Terminates the edit session but does NOT write out to the program file. Therefore, program changes made during the edit session are NOT recorded. The user will be prompted as to whether he actually wants to terminate in this manner.

CONTROL R

Refreshes the VT100 screen.

Edit Function Control Keys The control keys on the right-hand edge of the numeric pad provide the following control functions:

NEXT RUNG (Display the next relay)

This key displays the next relay or comment. If a signal name is input prior to the NEXT key, the next relay that outputs that signal name will be displayed. If the signal name is not found, the END OF DATA message will appear.

PREV RUNG (Display the previous relay)

This key displays the previous relay and comment. If a signal name is input prior to the PREV key, the previous relay that outputs that signal name will be displayed. If the signal name is not found, the START OF DATA message will appear.

SEARCH FORWARD (Search forward for signal name)

This command requires that a signal name be specified prior to the SEARCH FORWARD key. If the signal name is not found, the NO NEXT OCCURRENCE message will be displayed. This key searches for the next relay that contains a reference to the specified signal name and displays it.

SEARCH BACKWARD (Search backward for signal name)

This command requires that a signal name be specified prior to the SEARCH BACKWARD key. If the signal name is not found, the NO PREVIOUS OCCURRENCE message will appear. This key searches for the previous relay that contains a reference to the specified signal name and displays it.

NOTE

If further search of signal name is needed (forward or backward) this can be accomplished with the SEARCH SIGNAL NAME FORWARD (,) or SEARCH SIGNAL NAME BACKWARD (6) keys, without specifying signal name again.

SEARCH COMMENT FORWARD

This command will find the next occurrence of the comment rung entered through the CF or CB special functions.

SEARCH COMMENT BACKWARD

This command will find the previous occurrence of the comment entered through the CF or CB Special function.

END OF PROGRAM

This command allows the last rung to be displayed.

START OF PROGRAM

This command allows the first rung to be displayed.

FIND/SHOW RUNG#

Depressing this key by itself will allow the rung number of the rung currently displayed on the screen; entering a number before depressing this key will allow the numbered rung to be displayed.

START RUNG EDITOR

Allows the rung displayed on the screen to be edited.

Special Functions in SSFSS Editor The special functions are accessed by typing a function code on the alphanumeric keyboard, followed by the SPECIAL key (**"."** key on the keypad). The special functions are described in detail below:

P (Create Ladder Diagrams)

This function outputs the program in diagram form to a file named FILENAME.LIS. This file can later be printed using the VAX PRINT command to obtain a human-readable form of the program. The entire program will be output starting from the beginning.

PN (Create Signal Name)

This function will output a list of all signal names and hardware addresses used within the program to a file named FILENAME.NAM. This file can later be printed using the VAX PRINT command.

RN (Rename Signal)

You will be prompted for an old and new signal name. The new name will replace the old name in all relays.

AC (Add Comment)

Allows the comments in the program to be placed back into the file after a reverse compile has been accomplished. The comments are stored in the comment reintegration file (.CMT). When the editor is called after the Reverse Compile, there are no comments. YOU should count the number of rungs in the old listing up to the point where a new rung was added or an old rung was deleted. The diagram displayed must be the starting rung for which comments are to be added. Do an AC Special (.). This will give a prompt for the number of comments to be added. The user then enters a rung # (i.e., "4") or set of rung #s (i.e., "1;200"). When the carriage return is depressed, the comments will be added and the next rung that a comment was added to will be displayed.

D (Delete)

Deletes the displayed rung from the diagram.

DM (Delete Multiple Rungs)

Deletes multiple rungs. When the DM Special (.) is entered, the following prompt asking for the number of rungs appears:

NUMBER RUNGS:

The number specified, starting with the rung shown in the display, will be deleted, The rung following the last rung deleted will then be displayed.

CB (Search comment Backward1

Allows you to search for a comment in the backward direction. When the "CB Splecial (.)" is entered, you will be prompted as follows:

ENTER STRING

YOU enter the string to be- searched. When you depress the "Return" key, the previous usage of that string appears.

If no usage appears, the prompt following the prompt will appear:

NO PREVIOUS USAGE

If the string IS found, you can search for the next previous use of the string by using the SEARCH CMT BACKWARD (9) key. Each time this key is depressed, the previous usage of this string will be displayed until there is no previous usage. The NO PREVIOUS USAGE prompt will appear.

CF (Search Comment Forward)

You enter the string to be searched and when the "Return" key is depressed, the next usage of that string will be displayed.

If no usage is found, the following prompt will appear:

NO NEXT USAGE

If the string IS found, you can search for the next use of the string by usihg the SEARCH CMT FORWARD (PF3) key. Each time this key is depressed, the next usage of this string will be displayed until there is no next usage; then the following prompt appears:

NO NEXT USAGE

Relay Editing

Relay editing is initiated by use of the Edit or Insert Relay control functions (See Control Functions). The Insert function begins with a blank display. The Edit Function begins with the old relay diagram on the display.

Relay editing is divided into two parts: (1) editing the relay diagram, and (2) editing the signal names. These are described below.

Diagram Editing The most common diagram editing functions are accessed by use of diagram keys. An additional set of less freaqently used diagram functions are accessed by use of the alphanumeric keyboard in conjunction with the special key.

Diagram Keys The cursor control keys and the diagram keys on the numeric pad provide the following diagram editing function:

CONTROL-R
Repaint the screen

CONTROL-Y
Return to function mode without saving any data

HELP
Display help information

Cursor Control Keys
The UP, DOWN, LEFT and RIGHT cursor control keys (located above the alphanumeric keyboard) are used to adjust the position of the cursor within the 10 x 8 diagram grid.

RETURN Key
The RETURN key positions the cursor to the first grid position on the left side of the next line.

-1 [- Contact Normally Open
This key will enter a normally open contact and position the cursor to the next grid position. NOTE: Not valid in column 10.

-]/[- Contact Normally Closed

This key enters a normally closed contact and positions the cursor to the next grid position. NOTE: Not valid in column 10.

+
! Vertical short
+

This key enters a vertical short between the current grid position and the grid position vertically above it. NOTE: Not valid on line 1.

+

Vertical delete
+

This key deletes a vertical short. NOTE: Not valid in line 1.

+---+ Horizontal short

This key enters a horizontal short and positions the cursor to the right. It will also delete any horizontal symbol previously entered.

+ +

Horizontal delete. This key deletes any horizontal symbol and positions the cursor to the right.

DELETE

This key deletes any horizontal symbol and positions the cursor to the left.

-( ) Relay

This key enters a relay output coil. If the cursor is not in column 10, horizontal shorts will be generated across to column 10.
NOTE: Only valid on line 1.

-( ) Latch
LATCH

If this key is used on line one, it enters a latch set and output coil. If the cursor is not in column 9, horizontal shorts will be generated across to column 9. If the cursor is not on line one, a latch reset will be generated. If the cursor is not in column 10, horizontal shorts will be generated across to column 10.

-( )
ONE One Shot
SHOT

This key enters a one shot coil. If the cursor is not in column 10, horizontal shorts will be generated across to column 10. NOTE: Only valid on line one.

PRE Preset or Accumulator
ACC

This key is used with the counter and timer keys to build counters and timers. If this key is used on line one, it will enter a preset. If the cursor is not on line one, it will enter an accumulator. If the cursor is not in column 9, horizontal shorts will be generated across to column 9.

-( )
COUNT Counter
U/D

If this key is used on line one, you will be prompted for the type of counter as follows:

COUNTER TYPE U/D?

Dependent on the input, an UP counter or a DOWN counter output coil will be entered. NOTE: Only valid in column 10.

-( )
TIMER Timer
H/T/S

If this key is used on line one, you will be prompted for the type of timer as follows:

TIMER TYPE H/T/S/?

Dependent upon the input, a .01, .1 or 1 second timer output coil will be entered. If the cursor is not on line one, a timer reset will be entered. NOTE: Only valid in column 10.

DIAGRAM Edit Names/Edit Diagram
NAMES

This key toggles between names editing and diagram editing. Names editing is described in Signal Name Editing.

END EDIT Enter Diagram in file

This key is used to enter the diagram at the completion of editing. The Editor will verify the diagram for illegal short and open circuits. If errors are detected, a message will be displayed on the bottom line of the display; for example - MISSING VERTICAL SHORT, and the cursor will be positioned to the point on the grid where the error is located. When the diagram verifies correct, the editor will prompt for the next control function. See Control functions.

Other Functions

Other LDSSS functions are entered by use of the mnemonic name of the function on the alphanumeric keyboard followed by the special key. The following functions are supported at this time.

MNEMONIC	FUNCTION
ARITH	
ADD	Add Binary
SUB	Subtract Binary
CMPR	Compare
SHIFT/MOVE	
SHIFT	Shift
I/OR	Move I/O table to register
RI/O	Move register to I/O table
BIBCD	Binary to BCD convert
BCDBI	BCD to Binary convert
MCR/SKIP/END	
MCR	Master Control Relay
SKIP	Skip
ENDSW	End of sweep

MNEMONIC	FUNCTION
NOOP	No operation
SCREQ	CCM request
DPREQ	DPU request
SIGNED ARITH	
DPADD	Double Precision Add
DPSUB	Double Precision Subtract
ADDX	Extended Add
SUBX	Extended Subtract
MPY	Multiply
MPY	Multiply
DVD	Divide
AGTB	A Greater Than B
TABLE MOVE	
TTOD	Table-To-Destination
STOD	Source-to-Table
MVTBL	Move Table
LIST	
ATT	Add to Top of List
RFB	Remove from Bottom of List
SORT	Sort Ascending

DATA MOVE

MVAB	Move A to B
MVR8	Move Right 8 Bits
MVL8	Move Left 8 Bits
BLOCK	Block Move

MATRIX

AND	Matrix AND
IOR	Matrix Inclusive OR
EOR	Matrix Exclusive OR
INV	Matrix Invert
COMPARE	Masked Compare

BIT MATRIX

BSET	Set/Sense Bit
BCLEAR	Clear/Sense Bit
SHIFTR	Shift Right N Bits
SHTFTL	Shift Left N Bits
DOSUB	Do subroutine
RETURN	Return
SUSPEN	Suspend I/O
DIO/O	Do I/O
STATUS	Status

These functions are only valid on line one, or the top line of a reset string.

The SKIP, ENDSW, NOOP and RETURN functions are only valid in column one, and any other diagram input will be ignored.

Signal Name Editing Signal name editing is initiated by use of the DIAGRAM/NAMES key. Signal names are entered using the alphanumeric keyboard. The following different kinds of signal names are recognized.

1. Mnemonic Signal Name - Any string of 1 to 7 alphanumeric characters except those defined below. A mnemonic name must be assigned to a signal address, register address at compile time.

2. Signal Address - A String of the form INNNN, ONNNN, AINNNN or AONNNN (where NNNN is a numeric value of 1 to 1024).

3. Register Address - A string of the form RNNNN (where NNNN is a numeric value of 1 to 1024).

4. Constant value - A string of the form CNNNN (where NNNN is a numeric value of 1 to 9999 for timers and counters) and CNNNNN(where NNNNN is a numeric value of 1 to 16383 for extended functions).

Signal name editing is accomplished using the alphanumeric keyboard and the special keys described below.

Cursor Control Keys

The UP, DOWN, LEFT and RIGHT cursor control keys (located above the alphanumeric keyboard) are used to adjust the position of the cursor within the 10 X 8 diagram grid. The cursor will automatically position itself to the next valid input location in the grid (moving left to right and top to bottom).

DELETE Key

The DELETE key deletes a signal name.

Space Bar

The space bar is used to terminate a signal name or to position the cursor to the next valid input location.

Comment Editing

Comment editing is initiated by the Edit Comment control functions (See Control Functions). The Insert Comment function begins with a blank display. The Edit function begins with the old comment on the display.

The comment editor is similar to the VAX EDT editor, but supports only a subset of EDTS functions.

Comment editing is accomplished using the alphanumeric keyboard and the following special keys.

Cursor Control Keys

The UP, DOWN, LEFT and RIGHT cursor control keys (located above the alphanumeric keyboard) are used to adjust the position of the cursor on the display.

TAB Horizontal Tab

The cursor will tab to the next mod 8 position on the line.

RETURN Key

The return key positions the cursor to the first character position on the next line.

Control-R

Repaint the screen

Control-Y

Return to function mode without saving any data.

Comment Edit Function Keys

1. HELP (PF2)

   This key initiates a user help session.

2. FIND (PF3)

   This key positions the cursor to the start of the search string entered.

3. PAGE (7)

   This key displays the next page of the banner when comment editing the "START OF DATA".

4. BOTTOM (4)

   This key moves the cursor to the bottom of the screen.

5. TOP (5)

   This key moves the cursor to the top of the screen.

6. NEXT WORD (1)

   This key moves the cursor to the start of the next word.

7. EOL (2)

   This key moves the cursor to the end of the current line.

8. NEXT LINE (0)

   This key moves the cursor to the next line down, column 1.

9. INSERT TEXT (3)

   This key initiates and terminates insert mode. Insertion is from the cursor forward. The first 20 characters of the text ahead of the cursor are displayed at the bottom of the screen for reference.

10. DEL C (,)

    This key deletes the character that the cursor is on. If the cursor is at, or beyond the end of the line, the following lines are scrolled up.

11. DEL W (-)

    This key deletes from the cursor to the beginning of the next word on the line. If the cursor is at, or beyond the end of the line, the following lines are scrolled up.

12. DEL L (PF4)

This key deletes from the cursor to the end of the line. If the cursor is at, or beyond the end of the line, the following lines are scrolled up.

13. EXIT (ENTER)

This key exits comment editing mode and returns you to function mode.

Special Comments Special comments are used to control the format of the final documentation, and to define register values. The special comments are defined in detail below in reference to the CRT display.

Banner Page - The banner page appears at the head of the final documentation as created by the document generator.

Comment editing the start of data will display the first page of the banner. Auther, D.S. , revision , etc. are entered here. The second and third pages of the banner contain the general index.

The tilde character (~) simply marks the fields to be filled in. If the input data does not fill the field, or does not exist, the remaining tilde characters need not be erased.

The Page Key (7) scrolls through the three pages of the banner.

Sheet Assignments - New sections of program can be initiated by use of sheet assignments. To change sheets, use the following format on the first line of comment:

@_ _ _ _ 73 characters for description

This can be reasoned as start "This Section of Program" at sheet no. _ _ _ _. The four blank characters after the "@" must be filled, such as sheet 300 would be written 0300, then a space before the description (which is limited to 73 characters). This will also force the creation of a new document file.

NOTE: a maximum of 37 page changes is allowed,

The comment over the last end of sweep in the program should read

@ 9600 CROSS REFERENCE TABLE

Section Headers and Footers - Section headers and footers are used by the document generator to logically segment the final document.

A section header is a comment or part thereof that describes an upcoming section of the diagram.

A section header must come first in the comment if no sheet assignment in that comment, or next line after the sheet assignments. It is designated by a hat (^) in column 1 of the comment line and causes a page eject in the final documentation (.D##) file. The section header can be up to 23 lines in length and the first character of each line must be a hat (^).

Along with each section header goes a section footer. A section footer is a condensed version of the section header to go in the running foot of the final documentation. It is a maximum of two lines, 40 characters, in length, and is designated by a slash (/) in column 1. The running foot will contain the same section footer until replaced by a new section header, section footer pair. The section footer line must be the next line after the section header.

VALUE STATEMENT:

The format for the VALUE statement should be made available

Action: The format for the value statement is shown below:

VALUE: Mnemonic1 = Constant, Mnemonic2 = Constant, ...
VALUE: Mnemonic3 = Constant, Mnemonic4 = Constant, ...

Following are the valid mnemonics:

1. SINGLE PRECISION MNEMONIC.

   1. One to seven characters symbolic name. The register number for the symbolic name must be defined in the IOD file.

   2. A register number. For example: R0100, R0001, R0963, etc.

2. DOUBLE PRECISION MNEMONIC.

   1. "DPR" followed by one to seven characters symbolic name. For example, DPRXYZ, DPRA, etc. The register number for the symbolic name must be defined in the IOD file. For the examples written above, "XYZ", and "A" should be defined in the IOD file.

   2. A double precision register number. For example: DPR0001, DPR0104, DPR0935, etc.

Inputs

Old Source File - This file contains a ladder program already in existence. The editor will read the file and editing will begin from there.

Comment Reintegration File - This file is generated by the reverse compiler from old source code. It contains all the comments found in the old source, and is used to reintegrate comments.

outputs

The LDSSS editor may generate three different kinds of output files. They are described in detail below:

1. Source file (FILENAME.SRC) A source file contains an LDSSS program in source code form. This form is used as an input to the LDSSS Compiler. The program is compressed for archival storage and is not meant to be human readable.

2. List file (FILENAME.LIS) A listing file contains an LDSSS program in diagram form. In this form, a program can be printed in human readable form or input into the LDSSS documentation program. The list file can be generated by the special function command "P" special.

3. Names file (FILENAME.NAM) A Names file contains a list of all signal names used within the program. This file is useful for hardware address allocation and checking the used mnemonics. This program can be generated by use of the special function "PN" special.

RECOVERY

The LDSSS editor allows the user to recover an edit session lost due to the system or program malfunction. To recover an edit session, type:

```
$  RSE/RECOVER
```

CHAPTER 3
SERIES SIX COMPILER/REVERSE COMPILER

The Series Six compiler compiles LDSSS programs into an object code file that can be loaded and executed in the LDSSS CPU. It uses as input an LDSSS program in source code form (generated by the LDSSS editor) and a file that maps signal names to hardware addresses.

A further option of the program is to reverse compile object code back into source. This would be done to update programs to reflect changes made in the field using the LDSSS programming data terminal.

The compiler is part of a series of tools designed to support the LDSSS. The other tools and the overall support plan are described in the LDSSS plan document.

Inputs To Compiler

The LDSSS compiler requires as input two files: a source file and an I/O Definition file. These are described below:

Source File - (FILENAME.SRC) contains the LDSSS program in source code form. This file is generated using the LDSSS editor.

I/O Definition File - (FILENAME.IOD) contains a list of all signal names used in the LDSSS program and their associated hardware addresses.

Outputs From Compiler

The LDSSS compiler generates two output files: The object code file and an error file. These are described below:

Object File - (FILENAME.OBJ) contains the LDSSS program in object code form. In this form the LDSSS program can be loaded and executed in the LDSSS.

Register File - (FILENAME.REG) Contains the register values to be down loaded.

Error File - (FILENAME.ERC) contains a list of all errors detected by the compiler. This file can be displayed using the VAX EDT editor or printed out using the VAX EDT editor or printed out using the VAX PRINT command.

The compiler detects the following types of programming errors: (1) Signal names that do not have hardware addresses defined, and (2) Incorrect signal types. For example, use of an I/O signal where a register is required, (3) Addresses that do not have mnemonic assigned and are used in the program, (4) Contacts or coils with no mnemonic or address assigned, (5) Mnemonics that are duplicated in IOD file, (6) Lines that are incorrectly formatted in the IOD file, (7) Duplicate addresses found in IOD file.

The LDSSS editor checks the LDSSS program for compliance with the syntax rules for relay diagrams, so syntax errors should not be detected at compile time.

Inputs to Reverse Compiler

Object file (FILENAME.OBJ)
Old source file (FILENAME.SRC)
I/O Definition file (FILENAME.IOD)

Outputs from Reverse Compiler

New Source file (FILENAME.SRC)
Comment file (FILENAME.CMT)
Error file (FILENAME.ERC)

Operating Instructions

The compiler is run by typing RSC.

The compiler will then prompt for the compile direction (compile or reverse compile) as follows:

**** Series Six Compiler (Version 1.0)****

BATCH OR INTERACTIVE:

COMPILE OR REVERSE COMPILE (C/R)?

The compiler will then prompt for the file name of the program to be compiled as follows:

ENTER FILE NAME:

The name entered will be used for all input and output files. Use the first part of filename only--leave off the extension.

If the compile option is selected, the compiler will proceed to compile source to object with the inputs and outputs described in the paragraphs entitled "Inputs" and "Outputs".

If the reverse compile option is selected, the compiler will prompt for GENERATE COMMENT REINTEGRATION FILE?, then proceed to reverse compile object to source. The compiler will accept an Object file and a Names file and will produce a Source file and Error file. The Source file generated may later be read into the LDSSS editor to generate revised documentation.

At the end of the compilation, if no errors were detected, the compiler will display the message:

COMPILATION COMPLETE WITH 0 ERROR(S)

If errors were detected, the message would read for example:

COMPILATION COMPLETE WITH 5 ERROR(S)

CHAPTER 4
DOCUMENTOR

The purpose of the LDSSS Documentation utility is to generate documentation for the LDSSS Relay Diagrams on the VAX, and documentation for PDT debugging.

Operation

To run the documentation program, type RSD. The program will prompt you as follows:

Batch or Interactive: Type B for batch or I for interactive.

File Name: Type the source file name without any extension.

If you answered batch, the following questions will appear:

- continue or stop on requisition # mismatch
- SECOND SHIFT

Input Used

The documentation program uses three input files as follows: The Relay Diagram Listing file (.LIS), the Source Code File (.SRC), and the I/O Description file, (.IOD). The user will define a file name for each LDSSS. Each input file will have that file name, along with the appropriate file type as defined below.

1. Relay Diagrams Listing File (.LIS)

   This file contains the basic relay diagram file, along with associated comments and absolute rung numbers. File type is .LIS.

2. I/O Description File (.IOD)

   This file is generated by using the VAX EDT Editor.

   This file contains the input/output descriptions including: Signal Address, Signal Name, Device Type, location, sheet number and a signal description.

3. Source code (.SRC)

   This file contains the user program in compressed form. It is generated automatically when exiting the editor with the "SAVE" function. It is used to retrieve the banner page information.

Outputs

The program generates output documentor files, type D01 through D99. The content of this file is as follows:

1. The Banner page preceeds the entire document. It contains the LDSSS number, requisition number, customer, general index and running footer.

2. The Relay Data ladder diagram is printed with names appearing above signal addresses, comments preceeding their associated relay, and sheet of origin appearing below coils used as inputs. Sequential rung numbers appear below each coil. Section headers start a new page and the corresponding section headers start a new page and the corresponding section footer stays in the running foot until replaced or the end of the document is reached. Where used" data appears to the right of each coil. up to 15 locations are displayed. If more than 15 locations exist, a "C" is printed after the 15th. If more levels of logic exist for the same rung, each will add 15 more locations for "where used" data. If a "C" is encountered following the "where used" data on the last line of logic of a rung, you must consult te cross reference table at the end of the document for the rest. The lines from the I/O data file associated with the signals used on a sheet will be printed at the bottom of that sheet.

3. The Comprehensive Cross Reference Table is a compilation of all the "where used" data. Entries marked with an "--" show where the address is a coil (is in position 10). Other entries show the coil number and location of rungs where that address was an input: This is the last file of the documentor files.

   The table of contents will list the starting page numbers for each of the sections of the program. (A new section for each "@" in the editor program).

   The "Use Of The Cross Reference File" can be used to reference the place where the inputs, outputs, and registers are used in the program. From this table, the true outputs to the I/O that are not used can be detected. If it is not intended to use coils more than once, the table will help to find these usages.

Documentor error files allow run time errors to be reported on an error file, type .ERD.

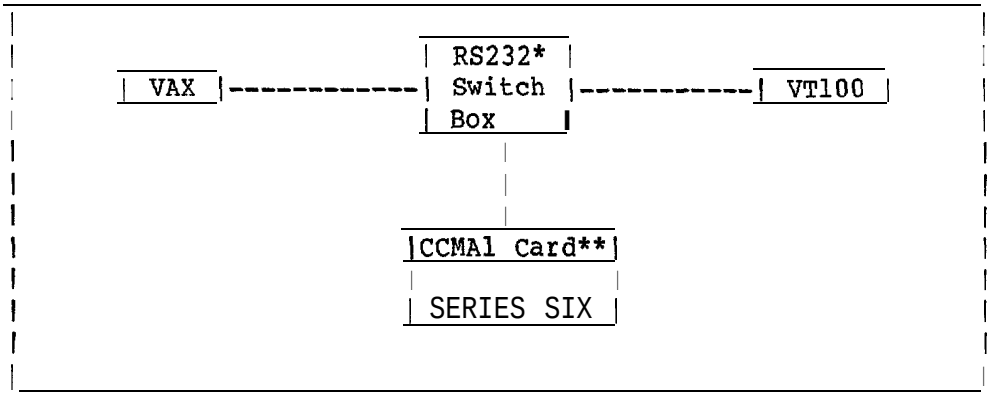
CHAPTER 5
CCMA1 LOADER

Inputs:
- Object file (FILENAME.OBJ)
- Register file (FILENAME.REG)

outputs
- Data to Series Six

Operation

To start the Loader, the RS232 switch box must be in the VT100 position. The loader is run by typing RSL.



The loader will then prompt for the file name of the program to be loaded as follows:

ENTER FILE NAME

The loader will then prompt for the 2-digit CPU ID (00 - 99) as follows:

Enter two digit CPU ID:

The loader will then prompt for the load direction (to or from the LDSSS CPU) as follows:

TO OR FROM CPU? (T/F)

If the user types "F" then the loader will prompt:

PROGRAM SIZE (1, 2, . . .31)

If the user types "T" then the program size is determined from the file, no prompt will appear.

* RS232 Switch - Giltronix Inc., Model GRS 232-S24AB.
**CCMA1 Card - General Electric Co., IC600CB514.

The loader will then prompt the operator to switch the VAX line over to the LDSSS CPU as follows:

SWITCH LINES

When the line has been switched, the RUN and ENABLE lights on the CPU card will go out to indicate that the program load has begun.

For a load to the LDSSS CPU, the file name must refer to an existing file that contains the object code for the program. The 1024 registers are loaded first, followed by the user program. User logic memory below the program remains unchanged.

For a load from the LDSSS CPU, a new file will be created (if the file name already exists a new version will be created). The entire user logic memory from the LDSSS CPU will be written to the file.

At the end of the program load, the RUN and ENABLE lights on the CPU card will come back and the enable light will blink 4 times to indicate that the load is complete. The VAX line must then be switched back to the VT100 position.

The operator must then depress RETURN to terminate the program. The program will respond with the following message:

LOAD COMPLETED

The load should be completed in under five minutes. In the event of an unsuccessful load, the DATA OK light on the CCM card will go out. The loader will abort after approximately 1 minute and an error message will be written to FILENAME.ERL. Reset the CCM card by depressing its top switch down and rerun the loader.

CHAPTER 6
INSTALLATION PROCEDURE FOR THE SERIES/6 VAX TOOLS

Following are the four software packages in the Series/6 VAX tools.

1. The Editor program
2. The Compiler/Reverse Compiler program
3. The Document Generator program
4. The CCMAl Loader

INSTALLATION PROCEDURE:

1. Create a "SSIX" directory.
2. Copy the files from the tape into this directory.
3. The system parameter VIRTUALPAGECNT must be set at 33824, so that the next command will work properly.
4. Type "@BUILD" to compile and link all the programs.
5. Add following commands in the system's login.com file.

   $ ASSIGN (actual disk name) SSIX$$SYSDISK

   $ ASSIGN SSIX$$SYSDISK:[SSIX] SIX$

   $ SIXEDIT :== $SSIX$$SYSDISK:[SSIX]SIXEDIT

   $ SIXDOC :== @SSIX$$SYSDISK:[SSIX]SIXDOC

   $ SIXCOMP :== @SSIX$$SYSDISK:[SSIX]SIXCOMP

   $ CCMAl :== RUN SSIX$$SYSDISK:[SSIX]CCMAl

6. To run the editor, the user will have to type : "SIXEDIT".
7. To recover an edit session, the user will have to type : "SIXEDIT/RECOVER".
8. To run the compiler, reverse compiler, the user will have to type : "SIXCOMP".
9. To run the document generator, the user will have to type : "SIXDOC".
10. To run the CCMAl loader, the user will have to type : "CCMAl".

APPENDIX
(FILE EXAMPLES)

.SRC FILE

```
B
B  AUTHOR:  GENERAL  ELECTRIC
B  D.S.  #:  4216
B  DRAWING #:  228B6825
B  STARTING  SHEET  #:  1
B  SHEET  NUMBER  INCREMENT:  1
B  ************************************************
B  *                                                *
B  *            SERIES  SIX  PROGRAM                *
B  *                    FOR                         *
B  *                                                *
B  *            SERIES  SIX  NO.  1                 *
B  *                    FOR                         *
B  *            GENERAL  ELECTRIC  COMPANY          *
B  *                                                *
B  ************************************************
B             REQUISITION No.  F47-835700
B
B
B
B
B
B                   GENERAL  INDEX
B  ************************************************
B  *     EQUIPMENT      * DRIVE NO. * ELEMENTARY     *
B  ************************************************
CVALUE:  R0890=08274 RO891 = 16710 R0892 = 19529 R0893 = 21077
CVALUE:  RO894=08261 R0895 = 08224 RO896 = 08255 R0897 = 08224


R162163  11
R164165C00896  166C00001  11
CSERIES  SIX  PROGRAM  RESET
R 43 13SSRESET
C^HYDRAULIC  PUMPS
C/HYDRAULIC   PUMPS
C/COPYRIGHT   1982
CPUMP 1 START
R  100173  3STARTP1  510169  10  5LOCAL81  #PMPMDA1  7PMPMDB1  5AO0415
R  1A0032  211002200  6A00221  7PP1STRT  9  6A09317  50LWSTLV  12PP1STRT
```

.LIS FILE

```
+
|^HYDRAULIC PUMPS
|/HYDRAULIC PUMPS
|/COPYRIGHT 1982
|PUMP 1 START
|
!O0173            LOCAL81 STOPPlL OFFPPl STPHYRE SSRESET      PPlDESI    PPlDESI
+--] [--+-------+--] [--+--] [--+--]/[--+--]/[--+--]/[--+-------+-------+-( )-6
!               !       !
!STARTPl 10169  :       :
+--] [--+--] [--+       +
!                       !
!PMPMDAl AO04l5         !
+--] [--+--] [--+-------+
!       !               !
!PMPMDBl!               !
+--] [--+               t
!                       !
IA0032Q                 :
+--] [--+-------+-------!
!                       !
!A00215 PMPMDCl         :
+--] [--+--] [--+-------!
:       :
!AO0216 !
+--] [--+
!
:
+
!
:A00320 A00220 A00221 A00217 OLWSTLV                                  PPlSTRT
+--] [--+--]/[--+--]/[--+--]/[--+--] [--+-------+-------+-------+-------+-( )-7
!       !               !
:       !PPlSTRT        !
t       +--] [--+-------+
!
!
+
!LOGIC TO AVOID STARTING THE PUMP SIMULTANEOUSLY
:
!STARTPl A00220 A00214                                                 PPSTRTG
+--] [--+--]/[--+--]/[--+-------+-------+-------+-------+-------+-------+-( )-8
!       !
!STARTP2!
+--] [--+
!       !
!TMPO70 !
t--l [--+ ,
:       !
!A00221 !
t--l [--+
```

.NAM FILE

****
SSRESET
PP1STR
PP2STR
STARTP1
STARTP2
PPSTRTG
STRTHRE
DTEMU15
PPSTRED
TMP070

STPHYRE

.IOD FILE

I/O Definition Line:

Signal 'type	:	Column l-2 (e.g., AI, _I, AO, _O, _R)
Signal Address	:	Column 3-6
Signal Name	:	Column 9-15
Device type	:	column 17-23
Device type	:	Column 25-31
Device location	:	column 25-31
Elementary sheet #	:	Column 33-39
Description	:	column 41-130

Blank Lines:

These will be ignored.

Comment Lines:

If there is a ";" in column 1 of any line, that will be treated as comment.

Example:

```
;
; VAXOUTI FOR REQN NO. F47-805700
;
;REQN NO. F47-805700                                          DRIVE NO. 80
;STATION = STATION 1              STATION DRIVE = 800
;ADDRESS  NAME       DEVTYP LOCATION SH NO    DESCRIPTION
AI0202     CDNRST     INT-STA                  COUNT DOWN RESET
 00200     PR5408P    INT-STA                  PR 54 OPENED W/I PRESSURE
A00201     PR480IP    INT-STA                  PR 48 OPENED WITH I PRESSURE
 10202     BTA58PR    INT-STA                  BAR TAIL APP. PR 58
A00203     PGPR23     INT-STA                  PG COUPLED TO PR 23 USED
 00204     PR38DIP    INT-STA                  PR 38 OPENED WITH I PRESSURE
 00205     PCI-12     INT-STA                  PCI-12 ACT.
 00206     PBJOGUP    INT-STA                  PS-1 P.B. JOG UP.
 00207     PBJOGDN    INT-STA                  PS-1 P.B. JOG DOWN
 00208     T1M2OS     INT-STA                  20 SEC. TIMER
 00209     T1M10S     INT-STA                  10 SEC. TIMER
 00210     20SECT     INT-STA                  20 SEC. TIMER
 00211     5SECTM     INT-STA                  5 SEC. TIMER
A00212     SICSTRD    INT-STA                  STARTING INTERLOCK FOR CAST READY
```

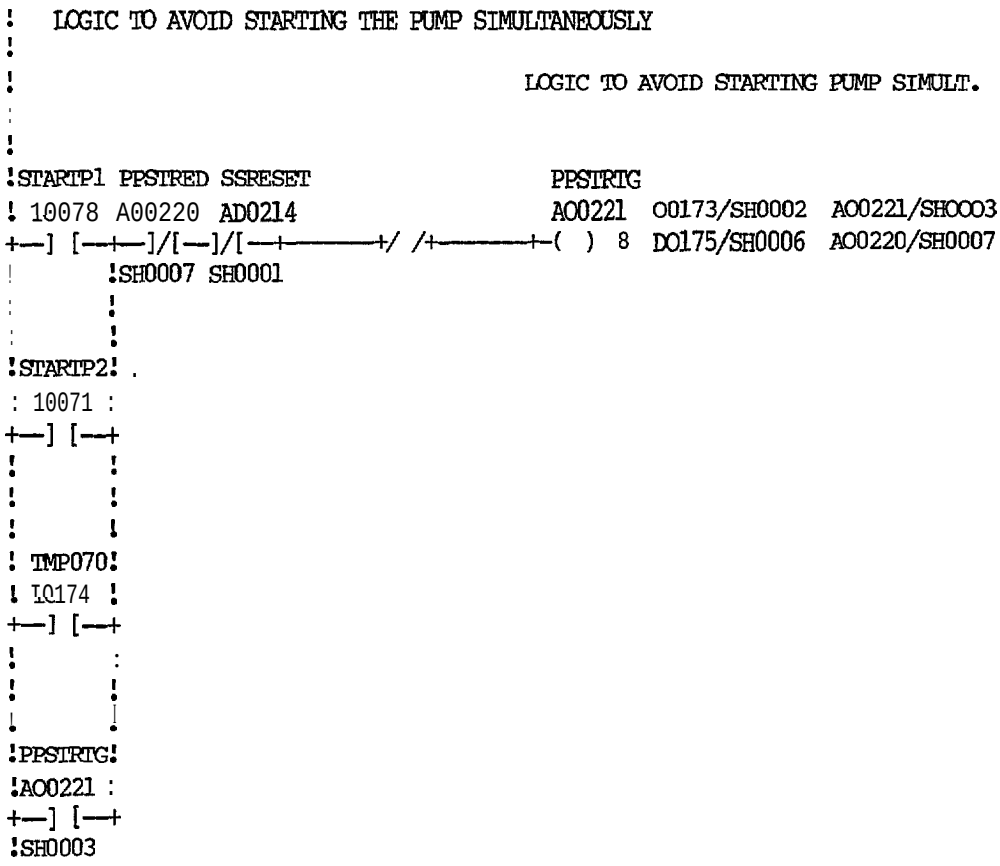
.OBJ FILE

```
00
86
00
50
FF
7D
31
0      2
F4
00
80
E5
33
51
F7
E5
33
00
50
30
F7
00
50
42
```

.REG FILE

089008274
089116710
089219529
089321077
089408261
089508224
089608224
089708224
089808224
089908224

.D01 FILE



ADDRESS	NAME	DEVTYP		LOCATION	SH NO	DESCRIPTION
10068	STARTP1	ILLPB	OP81	11AE		CIRCULATION PUMP #1 START
A00220	PPSTRED	INT-STA				STARTING PUMP LATCH
AO0214	SSRESET	INT-STA				SERIES SIX RESET
AO0221	PPSTRTG	INT-STA				LOGIC TO AVOID STARTING PUMP SIMULT.
10071	STARTP2	ILLPB	OP81	11AE		PUMP #2 START
IO174	TMP070	LSNC		11AM		HIGHER TEMP OF HTR HTR OFF

****

* * * * CROSS REFERENCE TABLE *

* DATE: 8-JAN-83 * REQUISITION # * GENERAL ELECIRIC *

* F47-835700 * DRIVE SYSTEMS DEPT **********

* AUTHOR * D.S. # * SALEM, VA USA * DRAWING # 228B6825 *

* GENERAL ELECIRIC CO. * 4216 * * NEXT SHEET - SH0166 SHEET - SH0165 *

****

.Dnn FILE

ADRS	NAME		WHERE USED DATA			
R0346		R0346/SH0088	R0347/SH0088			
R0347		R0347SH0088	A00250/SH0088			
R0348		A00250/SH0088				
R0350		R0350/SH0088	R0352/SH0089			
R0352		R0352/SH0089	O0417/SH0089			
R0400		A00325/SH0066	A00326/SH0066	R0402/SH0066	R0421/SH0067	
		A00882/SH0103	A00883/SH0103	A00884/SH0103	A00885/SH0103	
R0401		R0401/SH0067				
R0402		A00326/SH0066	R0402/SH0066			
R0404		A00326/SH0066	A00327/SH0066	A00328/SH0066		
R0620		R0541/SH0041	R0421/SH0067			
R0622		R0542/SH0041	R0550/SH0041	R0422/SH0067		
R0623		R0430/SH0067				
R0624		R0822/SH0041	R0460/SH0067			
R0626		R0558/SH0-041	R0448/SH0067			
R0628		R0561/SH0041	R0451/SHG0068			

```
**************************************************************************************************************
*                          *                          *                                  *                      *
*  DATE:  8-JAN-83         *  REQUISITION #           *  GENERAL ELECTRIC                *    CROSS REFERENCE TABLE *
*                          *      F47-835700          *  DRIVE SYSTEMS DEPT              ************************************
*       AUTHOR             *  D.S. #                  *  SALEM, VA USA                   *  DRAWING # 228B6825                *
*  GENERAL ELECTRIC CO.    *        4216              *                                  *  NEXT SHEET - SH0166  SHEET - SH0165 *
**************************************************************************************************************
```

GENERAL ELECTRIC COMPANY • DRIVE SYSTEMS DEPARTMENT
SALEM, VA. 24153

GENERAL ELECTRIC